

BUILDING LLM-POWERED APPLICATIONS · LECTURE 5

Building a Vector Storage

Turning text into geometry, so meaning becomes searchable.

Press  for fullscreen · navigate with  

Where we're going

The problem

- An LLM only knows its training data
- Your docs, PDFs, and notes are invisible to it
- Keyword search misses meaning — *"car"* ≠ *"automobile"*

Today's objectives

- Turn text into vectors
 - Rank by cosine angle
 - Chunk + index for fast retrieval
- This is the retrieval half of RAG.

Which TWO sentences are closest in MEANING? A: "How do I reset my password?" B: "The cat sat on the mat." C: "I forgot my login and can't sign in." D: "Reset the circuit breaker in the garage."

A and C — same idea, almost no shared words ✓

A and D — both contain the word 'reset'

B and C — both are short everyday sentences

A and B — both are questions about a problem

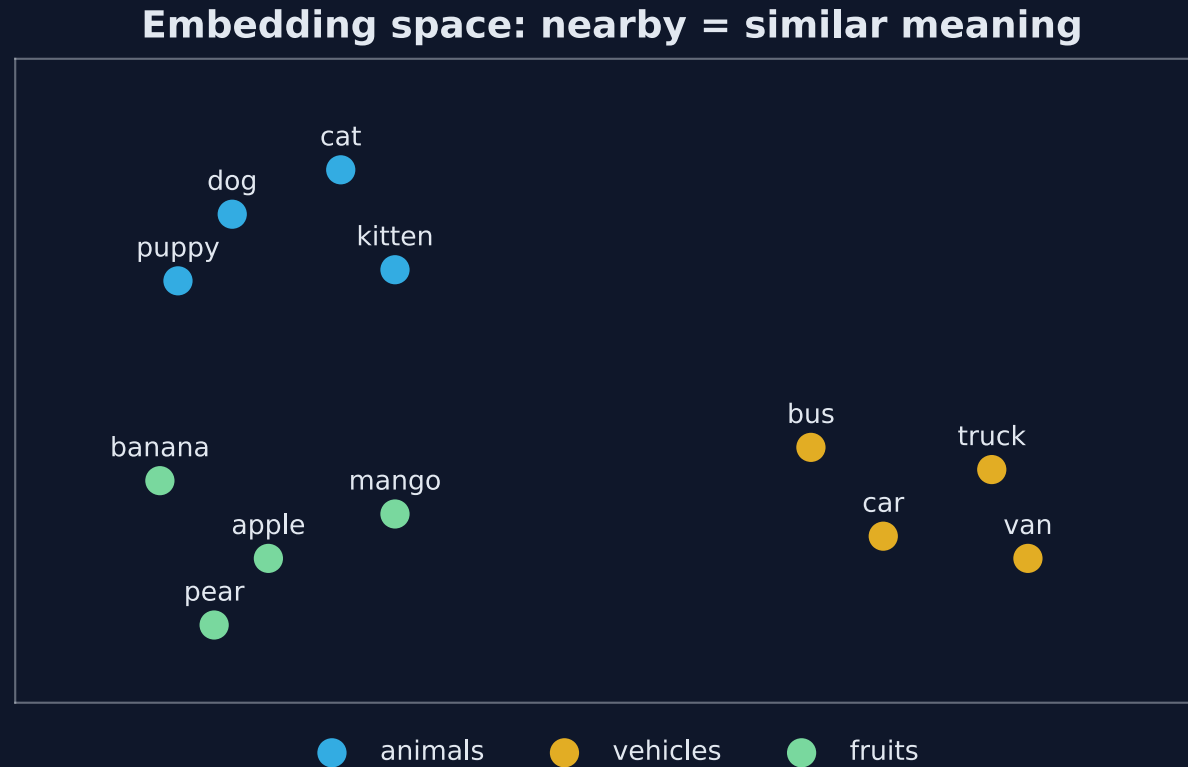
Meaning as geometry

An **embedding model** maps a piece of text to a fixed-length list of numbers — a **vector**. Texts that *mean* the same thing land close together in this space.

"a cat sat on the mat" → [0.12, -0.04, 0.88, ...] (1024 for our bge-m3 — size varies by model: 384–3072)

Direction encodes meaning. $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ works because related concepts point in related directions.

Embeddings put meaning into geometry



An embedding model maps text to a point so that **similar meanings land close together** — animals near animals, vehicles near vehicles.

First, a toy embedding: count the words

Before we reach for a real embedding model, we'll approximate a vector by **counting words** — a **bag-of-words** count vector.

Tiny vocabulary [cat, sat, mat, dog] :

"the cat sat on the mat" → [1, 1, 1, 0]

"a dog barked" → [0, 0, 0, 1]

A stand-in we'll feed to **cosine** and the toy demo next. Later we swap it for a trained model — and see why bag-of-words falls short.

Cosine similarity

To compare two vectors we measure the **angle** between them, not the distance. Same direction = same meaning, regardless of length.

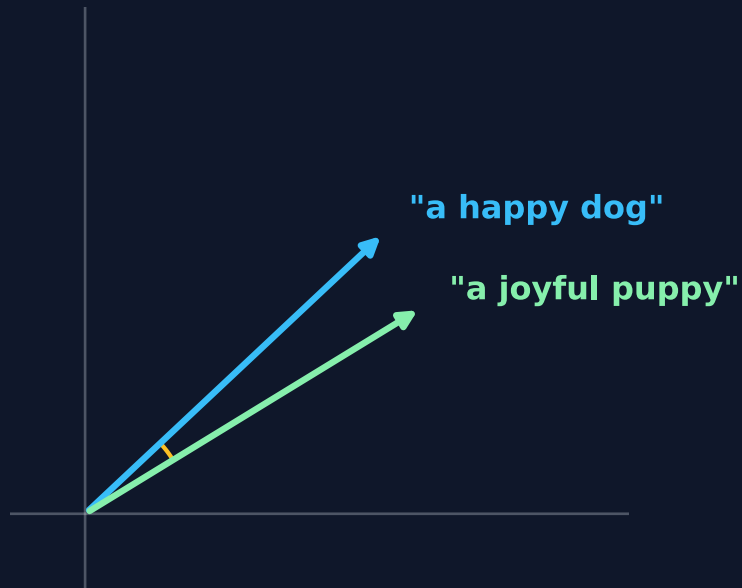
$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

where $a \cdot b = \sum_i a_i b_i$ is the dot product and $\|a\| = \sqrt{\sum_i a_i^2}$ is the length (norm).

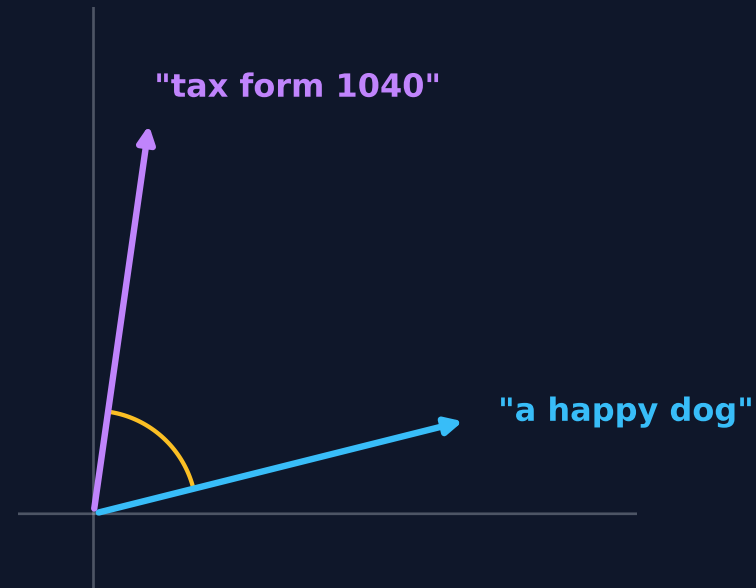
+1 = identical direction · **0** = unrelated (orthogonal) · **-1** = opposite. Nearest neighbor = highest cosine.

Cosine similarity = the angle, not the length

similar meaning → small angle
 $\cos \theta \approx 0.98$



unrelated meaning → large angle
 $\cos \theta \approx 0.34$



Cosine compares **direction**, not **magnitude** — same direction ≈ 1 , perpendicular ≈ 0 . Length (how long the document is) doesn't move it.

Cosine, worked by hand

Take two tiny vectors and turn the formula into arithmetic: $a = [1, 0, 1]$ and $b = [1, 1, 0]$.

Numerator: dot product

$$a \cdot b = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 1$$

Denominator: the two norms

$$\|a\| = \sqrt{1^2 + 0^2 + 1^2} = \sqrt{2}, \text{ and} \\ \text{likewise } \|b\| = \sqrt{2}.$$

$$\cos(a, b) = \frac{1}{\sqrt{2} \cdot \sqrt{2}} = \frac{1}{2} = 0.5$$

They share one of two directions — a 60° angle. This exact case is your HW3 sanity check:

```
cosine_similarity([1,0,1],[1,1,0]) == 0.5
```

PREDICT → RUN → MODIFY

Before you run it

The next slide ranks four documents against the query `"where did the cat sit"` with bag-of-words + cosine.

Commit to an answer:

- Which document ranks #1?
- What does `"kittens are baby cats that purr"` score — high, low, or 0.000?

Write a number, then run.

Cosine Toy

```
# A toy "embedding": bag-of-words vectors over a tiny vocabulary, then cosine  
# similarity in PURE Python (no libraries). Real apps use an embedding model  
# (next slide runs bge-m3) – but the math is exactly this. Press Ctrl/Cmd+Enter.  
import math  
  
docs = [  
    "the cat sat on the mat",  
    "a dog barked at the mailman",  
    "kittens are baby cats that purr",  
    "the stock market fell sharply today",  
]  
  
query = "where did the cat sit"  
  
# Build a shared vocabulary, then a count vector per text.  
def tokens(text):  
    return text.lower().split()
```

Normalize

*# Normalize a vector to length 1, and cosine collapses to a single dot product.
This is why some indexes ask you to normalize on insert: a plain inner product
then IS cosine. (pgvector's <=> normalizes for you.) Press Ctrl/Cmd+Enter.*

```
import math
```

```
a = [1, 0, 1]
```

```
b = [1, 1, 0]
```

```
def cosine(a, b):  
    dot = sum(x * y for x, y in zip(a, b))  
    na = math.sqrt(sum(x * x for x in a))  
    nb = math.sqrt(sum(y * y for y in b))  
    return dot / (na * nb) if na and nb else 0.0
```

```
def normalize(v):
```

In the toy demo, 'kittens are baby cats' scored 0.0 against the query about a cat. Why?

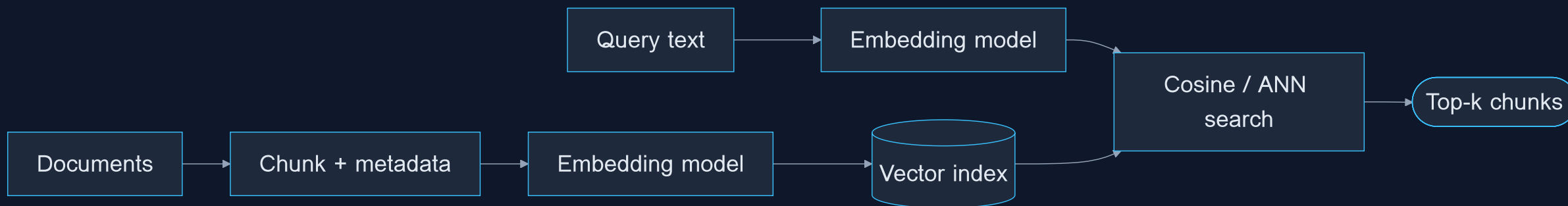
Cosine similarity is broken for short texts

Bag-of-words matches exact words, so 'cat' and 'kittens/cats' share no token ✓

The query was too long

The vectors were not normalized

Pipeline



Real embedding models

Our toy counts words. A trained **embedding model** reads the whole sentence and outputs a dense vector where *synonyms* land close — so `cat` and `kitten` are neighbors with no shared word.

Our class model

`bge-m3` (1024-dim, multilingual — EN + Thai), on the class proxy. Runs in the sandbox →

Others

Hosted: OpenAI `text-embedding-3-small`, Cohere · Local: `sentence-transformers` (`all-MiniLM-L6-v2`).

Same OpenAI call, just `model="bge-m3"`; real pipelines batch:

```
embeddings.create(input=[chunk1, chunk2, ...]) → read each from  
resp.data[i].embedding.
```

Pick one model and **stick with it** — you can only compare vectors from the same model.

PREDICT → RUN → EXPLAIN

Watch the kitten line

The next slide re-runs the *same* ranking — query `"where did the cat sit"` over the same four docs — but with the real model `bge-m3` instead of word counts.

Bag-of-words gave `"kittens are baby cats that purr"` a flat **0.000** (no shared word). Predict:

- Does the kitten line now beat the unrelated **stock-market** line?
- Will it outrank `"the cat sat on the mat"`, or land just under it?

Same cosine, same docs — only `vectorize()` changed.

Real Embeddings Run

```
# The SAME search as the bag-of-words toy, but with the real class embedding  
# model `bge-m3`. Watch the kitten sentence climb off zero – it shares NO words  
# with the query, yet the model knows kittens are cats. Press Ctrl/Cmd+Enter.  
import math  
from openai import OpenAI  
  
client = OpenAI() # in the sandbox the proxy URL + key are already set  
  
docs = [  
    "the cat sat on the mat",  
    "a dog barked at the mailman",  
    "kittens are baby cats that purr",  
    "the stock market fell sharply today",  
]  
query = "where did the cat sit"
```

Multilingual

```
# bge-m3 is MULTILINGUAL – the reason we chose it. It puts a sentence and its
# translation near each other with NO shared characters. Press Ctrl/Cmd+Enter.
import math
from openai import OpenAI

client = OpenAI()

texts = [
    "how do I reset my password",          # EN query
    "ฉันจะรีเซ็ตรหัสผ่านได้อย่างไร",      # SAME meaning, Thai
    "the cat sat on the mat",              # unrelated EN
    "ตลาดหุ้นร่วงลงอย่างหนักวันนี้",      # unrelated Thai (stock market fell)
]

def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
```

In the bge-m3 demos, even UNRELATED lines scored $\sim 0.3-0.5$ (not 0). So how should you decide which chunk is the match?

RANK the candidates and take the highest cosine — absolute scores are crowded, not calibrated ✓

Keep only chunks scoring above a fixed 0.8 threshold; discard the rest

Average all the cosines and return whatever beats the average

Trust any score above 0.0 as a real match

Batch Embeddings

```
# Batch your embedding calls. The API takes a LIST and returns one vector per
# item, in order. A 300-page PDF is thousands of chunks – send a few hundred per
# call, not one request each. resp.data[i] lines up with input[i]. Run it.
from openai import OpenAI

client = OpenAI()

chunks = [
    "To reset your password, open account settings.",
    "Login problems are fixed from the security page.",
    "Kittens are baby cats that like to purr.",
    "The stock market fell sharply today.",
]

# ONE call embeds all four. data[i] corresponds to chunks[i].
resp = client.embeddings.create(model="bge-m3", input=chunks)
```

Chunking & metadata

You don't embed a whole PDF as one vector — you split it into **chunks** and embed each. Chunk size decides what a single "hit" returns.

Size & overlap

- **Too big** → vague vector, noisy context
- **Too small** → loses surrounding meaning
- ~200–500 tokens with **10–20% overlap** so a sentence isn't cut at a boundary

Attach metadata

Store `source`, `page`, `section`, `date` with each chunk. You'll need it to cite sources and to filter searches in Lecture 8.

You're about to split 300-page PDFs for retrieval. Predict: what happens if you embed each page as ONE big chunk?

Bigger is always better — more text per vector means a more accurate match

The vector blurs many topics together, so it matches everything vaguely and nothing sharply ✓

Embedding fails — models reject inputs longer than one sentence

Retrieval gets faster because there are fewer vectors to search

Three ways to split the same paragraph

Paragraph (14 words): To reset your password open account settings then choose security and follow the link emailed to you

Fixed-size, no overlap

[w1..w6] [w7..w12] [w13..w14]

"the link emailed to you" is stranded
— split from "reset your password."

Fixed-size + overlap (2)

[w1..w6] [w5..w10] [w9..w14]

Edges shared → the reset action and the link co-occur. Costs ~30% more chunks.

Sentence / paragraph boundaries: each chunk a complete thought — but sizes vary wildly, so you re-pack up to a token budget.

THINK · PAIR · SHARE

Metadata earns its keep twice

A stored record is more than text + vector. Discuss with your neighbor — for a Q&A bot over the course PDFs:

Citations

If the bot answers from a chunk, which fields let you show "from handbook.pdf, page 7"? Why does a user need that?

Filtering

A student asks only about *Lecture 5*. Which field would you filter on before ranking — and how does that help speed and accuracy?

A record: `{id, text, source, page, section, date, embedding}`. Filtering before ranking is the seed of hybrid search (Lecture 8).

You're building Q&A over 300-page course PDFs (HW3). A question's answer is usually 1–3 sentences. Best default chunking?

~300 tokens per chunk with ~15% overlap, split on paragraph boundaries ✓

One chunk = the whole PDF, so no context is ever lost

One chunk per sentence, zero overlap, for maximum precision

~2000 tokens per chunk so each chunk is self-contained

In pgvector, ` $\leq$ ` returns cosine DISTANCE, where smaller means closer. A student's query returns the LEAST relevant chunks. Which mistake explains that?

They added DESC to the ORDER BY — that puts the farthest (largest distance) chunks first ✓

Distance is lower-is-closer, so the ordering is already correct and the bug is elsewhere

They should switch to cosine similarity in SQL and sort ascending

pgvector can't rank — you must pull all rows and rank in Python

Chunk Code

*# Chunking with overlap + metadata, in pure Python. Real pipelines embed each
chunk and upsert it into pgvector; here we just print what gets stored.*

```
def chunk(text, size=12, overlap=2): # overlap ~15% of size, matching 10-20%
    words = text.split()
    step = size - overlap
    out = []
    for start in range(0, len(words), step):
        piece = words[start:start + size]
        if piece:
            out.append(" ".join(piece))
        if start + size >= len(words):
            break
    return out
```

```
document = (
    "Vector storage turns text into embeddings. Each chunk is embedded into a "
    "vector. The vectors are stored in an index. At query time we embed the "
```

Chunk Overlap Run

```
# Worked split of ONE paragraph, three ways. Watch where the answer survives.  
# Pure Python, no libraries. Press Ctrl/Cmd+Enter, then change `overlap`.  
def fixed(words, size, overlap=0):  
    step = size - overlap  
    out = []  
    for start in range(0, len(words), step):  
        piece = words[start:start + size]  
        if piece:  
            out.append(piece)  
        if start + size >= len(words):  
            break  
    return out  
  
text = ("To reset your password open account settings then choose "  
        "security and follow the link emailed to you")  
words = text.split()
```

Your HW3 index holds a few thousand course-PDF chunks. The brute-force loop (cosine against every chunk) searches in milliseconds. Should you add an ANN index now?

No — a linear scan is fine into the low tens of thousands; add an index only when it actually gets slow ✓

Yes — always build an HNSW index before any search, even for 10 vectors

Yes — brute force gives wrong answers above 1000 vectors

No — ANN indexes only work on GPUs, which HW3 doesn't have

Vector Store

```
# A tiny in-memory vector store in PURE Python: add() chunks, then search() for  
# the top-k nearest by cosine. Same BoW vectorizer + cosine as before – swap in a  
# real embedding model and this is exactly how a vector store works.
```

```
import math
```

```
# --- toy embedding: bag-of-words over a shared vocabulary ---
```

```
def tokens(text):
```

```
    return text.lower().split()
```

```
def cosine(a, b):
```

```
    dot = sum(x * y for x, y in zip(a, b))
```

```
    na = math.sqrt(sum(x * x for x in a))
```

```
    nb = math.sqrt(sum(y * y for y in b))
```

```
    return dot / (na * nb) if na and nb else 0.0
```

```
class VectorStore:
```

What brute force costs

The toy `search()` compares the query to **every** stored vector — that's brute-force kNN, $O(N)$ per query.

The arithmetic

- 1 query $\times$ N docs $\times$ 1024 multiply-adds each
- 4 docs $\rightarrow$ instant
- 1,000,000 docs $\rightarrow \sim 10^9$ ops *per query*

When it's fine

Don't index too early. A linear scan is fast into the low tens of thousands of chunks — a few thousand course-PDF chunks search in milliseconds.

Past $\sim 10k$ – $100k$ vectors (or sub-ms latency under load), the scan hurts and a real **index** earns its complexity.

Why a vector *index*?

Our toy compares the query to **every** document — that's brute-force kNN. Fine for 4 docs; at a million vectors it's far too slow per query.

ANN

Approximate Nearest Neighbor: trade a little recall (you may miss the true #1) for huge speed. Build a smart structure (graph / clusters) once, query it fast.

In practice

`pgvector` — vectors inside Postgres (what we use) · Pinecone, Qdrant, Weaviate — managed services.

Brute force is fine into the low tens-of-thousands of vectors; reach for an index beyond that. A vector store = the vectors + their metadata + an index that answers "give me the top-k nearest" quickly.

How ANN skips the scan

Brute force visits all N vectors. ANN builds a structure *once*, then at query time touches only a small, promising slice of it.

Graph (HNSW)

Vectors are nodes linked to near neighbors. Start anywhere and greedily hop toward the query, visiting a few hundred nodes — not a million.

Clusters (IVFFlat)

Group vectors into buckets. Find the few buckets closest to the query, then scan only those.

Both are pgvector index types on the `embedding` column. More search effort → higher recall; less → faster. You tune that knob.

An Approximate Nearest Neighbor (ANN) index makes search far faster than brute force. What do you give up in return?

A little recall — it may occasionally miss the true #1 neighbor ✓

Nothing; ANN is exact and strictly better than brute force

The ability to store metadata alongside vectors

Cosine similarity; ANN only supports Euclidean distance

pgvector: search in SQL

The `pgvector` extension adds a `vector` column type and distance operators to Postgres — so retrieval is just a query against your existing database.

```
CREATE EXTENSION vector;
CREATE TABLE chunks (
  id bigserial, content text,
  source text, embedding vector(1024)); -- bge-m3 → 1024

-- <=> is cosine distance; smaller = closer
SELECT content FROM chunks
ORDER BY embedding <=> $query_vec
LIMIT 5;
```

Metadata (`source`, page, date) lives in the same row — filter *and* rank in one query.

From toy to HW3

HW3 re-implements these primitives as **auto-graded** functions. Same math — but the contract is pinned down. Match it exactly:

`cosine_similarity(a, b)`

round(value, 4); all-zeros vector →

`0.0`. Check: `cos([1, 0, 1], [1, 1, 0]) == 0.5`.

`chunk_text(text, size)`

Fixed-size, non-overlapping words (the §6 toy overlaps on purpose — HW3 does not). `" "` → `[]`.

`top_k(query, docs, k)`

Return **indices** (not texts), nearest first; ties → **lower index** wins. Sort on `(-score, index)`.

Same embed → score → sort → top-k shape as the toy store — HW3 just pins the return types for the grader.

pgvector's `<=>` operator returns cosine DISTANCE, where smaller = closer. To get the nearest chunks you wrote `ORDER BY embedding <=> $q`. Which direction is correct?

Ascending (default) — smallest distance is the nearest neighbor; take the top LIMIT rows ✓

Descending — the largest `<=>` value is the closest match

It doesn't matter; `<=>` already sorts by similarity

Sort by `embedding <=> $q DESC` to mimic highest-cosine-first

Distance vs similarity — the sign flip

Cosine **similarity** is higher = better (max +1). pgvector's `<=>` is cosine **distance** =

$$1 - \cos(a, b)$$

— lower = better (min 0). Mix them up and you sort backwards.

You computed	Best match is the...	Ordering
cosine similarity	highest	<code>sort(reverse=True)</code> · <code>ORDER BY ... DESC</code>
cosine distance (<code><=></code>)	lowest	<code>sort()</code> · <code>ORDER BY ... ASC LIMIT k</code>

For `a=[1, 0, 1]`, `b=[1, 1, 0]` both happen to be `0.5` — a coincidence of that pair. Trust the **rule's direction**, not the number.

Batch embeddings — keep `data[i]` aligned

One call per chunk is slow and burns rate limit. Pass a [list](#); the API returns one vector per item, *in the same order*. The contract you must not break: `resp.data[i].embedding` is the vector for `input[i]`.

```
resp = client.embeddings.create(model="bge-m3", input=chunks)
for i, chunk in enumerate(chunks):
    vec = resp.data[i].embedding    # data[i] <-> chunks[i]
    assert len(vec) == 1024        # bge-m3 is always 1024-dim
```

Don't sort `resp.data`, **don't** filter inputs after the fact. A 300-page PDF → batch a few hundred chunks per call, not thousands of requests. [Same rule at any batch size.](#)

Symptom → fix

Symptom	Fix
Everything "kind of related," nothing clearly best	Chunks too big → 200–500 tokens; split on boundaries
Answer sentence never retrieved, even verbatim	It was cut at a boundary → add 10–20% overlap
pgvector returns the <i>worst</i> matches first	<code><=></code> is distance → <code>ORDER BY ... ASC</code>
<code>top_k</code> returns the wrong texts	<code>data[i]</code> misaligned → read back in input order
Related texts score ~0.3, look "unrelated"	Spaces are crowded → rank, don't threshold
<code>ZeroDivisionError</code> on a chunk	Empty chunk → return <code>0.0</code> when a norm is 0
Cross-source results leak in	Pre-filter on <code>source</code> before ranking

Most retrieval bugs are one of these seven — lead with the symptom you observe.

In your HW3 Mini-RAG, retrieval returns garbage. Which ONE mistake most likely breaks it?

You embedded documents with one model but the query with a different model ✓

You used cosine similarity instead of Euclidean distance

You stored metadata (source, page) alongside each chunk

You normalized your vectors to length 1 before indexing

Recap & what's next

You can now

- Explain text → embedding as meaning-geometry
- Compute cosine similarity by hand
- Chunk a document with overlap + metadata
- Embed text with `bge-m3` and store it in pgvector

Read the lecture notes, then start HW3 (Mini-RAG over course PDFs).

Next lecture

Lecture 6

Retrieval-Augmented Generation — retrieve the nearest chunks, stuff them into the prompt, and let the LLM answer *grounded* in your docs.

